

Traffic Signal Violation Detection System

Technical Documentation & Logic Specification

*Zone-based, signal-fused detection of traffic signal violations: classifies vehicles in the detected zone during a red signal as **stop-line** or **red-light** violations.*

1 Overview

The **Traffic Signal Violation Detection (TSVD)** module follows a sequential decision-making pipeline to classify traffic violations based on the traffic signal state, vehicle position, and previously estimated vehicle speed. The overall flowchart is illustrated in Figure 1.

Initially, the module receives a pre-processed and enhanced CCTV frame from the image enhancement stage. The image is then logically divided into two regions: a *Normal Zone*, representing the area before the stop line, and a *Detection Zone*, representing the area beyond the stop line where violations are evaluated.

Unlike conventional vision-based systems, the current traffic signal state is not inferred from the CCTV image but is directly obtained from the traffic control system. This eliminates errors caused by poor illumination, occlusions, adverse weather conditions, and camera perspective.

The first decision block checks whether the current signal state is *Red*. If the signal is Green or Yellow, every detected vehicle is immediately classified as *No Anomaly*, and no further processing is required.

If the signal is Red, the module verifies whether the detected vehicle lies inside the predefined Detection Zone. Vehicles remaining outside this region are considered compliant with traffic regulations and are therefore classified as *No Anomaly*.

For vehicles located inside the Detection Zone, an anomaly is identified and the previously computed vehicle speed is evaluated. If the vehicle speed is greater than zero, the system classifies the event as a *Red-Light Violation*, indicating that the vehicle has crossed the stop line while continuing to move during a red signal. Conversely, if the vehicle speed is equal to zero, the event is classified as a *Stop-line Violation*, indicating that the vehicle has crossed the stop line and stopped outside the marked area.

Thus, the proposed pipeline performs traffic violation detection through a deterministic sequence of logical decisions without requiring additional image processing or deep learning inference. Since each vehicle undergoes only a constant number of comparisons, the overall *computational complexity remains linear with respect to the number of detected vehicles*, making the system suitable for real-time intelligent traffic monitoring applications.

2 Stage-by-Stage Description

The pipeline takes the **pre-processed, enhanced frame** I_{enh} as its input (image acquisition and enhancement are assumed already complete upstream).

2.1 Spatial Zoning (Normal vs. Detected Zone)

The enhanced frame is partitioned into two disjoint regions using the stop-line position as the separating boundary:

$$I_{enh} = Z_N \cup Z_D, \quad Z_N \cap Z_D = \emptyset$$

where Z_N is the *Normal Zone* (before the stop line, legal waiting area) and Z_D is the *Detected Zone* (at/beyond the stop line, into the intersection).

A vehicle, represented by its bounding-box centroid (x_c, y_c) , is determined to be in the detected zone using a point-in-polygon / half-plane test. For a stop-line modeled as a line $ax + by + c = 0$:

$$\text{Zone}(x_c, y_c) = \begin{cases} Z_D & \text{if } ax_c + by_c + c > 0 \\ Z_N & \text{if } ax_c + by_c + c \leq 0 \end{cases}$$

2.2 Signal-State Fusion

Unlike the spatial zoning, the traffic signal color S is **not inferred visually** from the camera frame; it is supplied directly by the control room as ground truth:

$$S \in \{\text{Red, Yellow, Green}\}$$

This avoids ambiguity/errors from signal-light recognition under glare, occlusion, or low-light conditions, and keeps the violation decision logic deterministic and auditable.

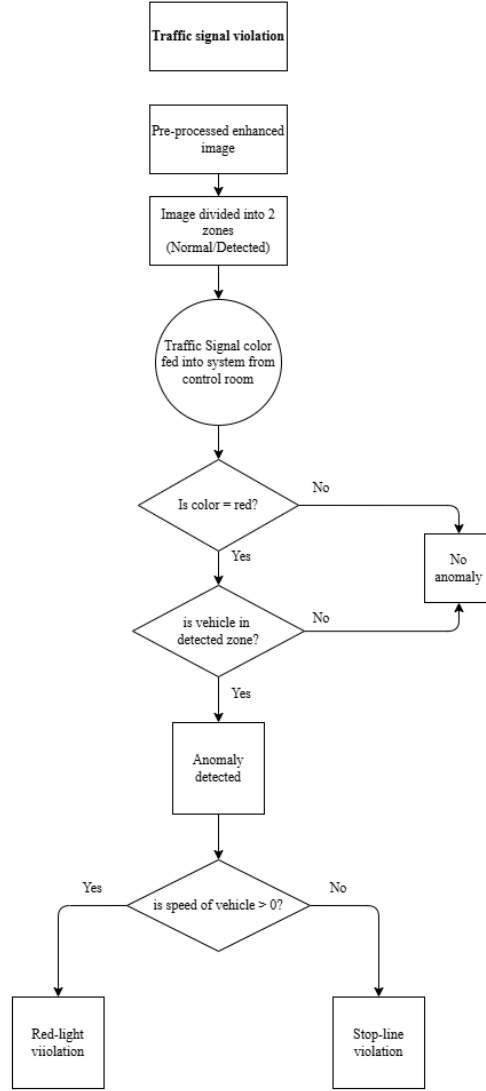


Figure 1: Traffic Signal Violation Flowchart

2.3 Violation Classification Logic

The core decision logic is a nested conditional evaluated per detected vehicle, per frame:

$$\text{Anomaly} = \begin{cases} \text{True} & \text{if } S = \text{Red} \wedge \text{Zone}(x_c, y_c) = Z_D \\ \text{False} & \text{otherwise} \end{cases}$$

When an anomaly is flagged, the vehicle's instantaneous speed v disambiguates the violation type:

$$\text{Violation Type} = \begin{cases} \text{Stop-line violation} & \text{if } v = 0 \\ \text{Red-light violation} & \text{if } v > 0 \end{cases}$$

Interpretation:

- **Stop-line violation:** the vehicle has advanced into the detected (restricted) zone but is currently stationary – e.g., it crept over the stop line and halted.
- **Red-light violation:** the vehicle is in the detected zone *and* still moving while the signal is red – an active light-running event.

3 Mathematical Formulation

3.1 Combined Decision Function

Vehicle speed v (and hence motion state) is assumed already computed upstream by the tracking module; let $M = [v > \epsilon]$ denote the moving flag. The full pipeline reduces to a single composite predicate per tracked vehicle i at frame t :

$$\text{Decision}_i^{(t)} = \begin{cases} \text{No anomaly} & S^{(t)} \neq \text{Red} \\ \text{No anomaly} & S^{(t)} = \text{Red}, \text{Zone}_i^{(t)} = Z_N \\ \text{Stop-line violation} & S^{(t)} = \text{Red}, \text{Zone}_i^{(t)} = Z_D, M_i^{(t)} = 0 \\ \text{Red-light violation} & S^{(t)} = \text{Red}, \text{Zone}_i^{(t)} = Z_D, M_i^{(t)} = 1 \end{cases}$$

3.2 Boolean / Truth-Table Representation

Let $R = [S = \text{Red}]$, $D = [\text{Zone} = Z_D]$ (Iverson brackets). The classification reduces to:

R	D	M	Outcome
0	–	–	No anomaly
1	0	–	No anomaly
1	1	0	Stop-line violation
1	1	1	Red-light violation

This is logically equivalent to:

$$\text{Violation} = R \wedge D, \quad \text{Type} = \begin{cases} \text{Stop-line} & \text{if } \neg M \\ \text{Red-light} & \text{if } M \end{cases} \quad (\text{given Violation} = 1)$$

4 Algorithmic Summary

Listing 1: Per-frame violation classification

```

1 EPSILON = 0.3 # m/s, motion threshold (tunable)
2
3 def get_zone(cx, cy, a, b, c):
4     """Half-plane test against the stop-line: ax + by + c = 0"""
5     return "DETECTED" if (a * cx + b * cy + c) > 0 else "NORMAL"
6
7 def classify_violation(signal_color, cx, cy, speed, stop_line):
8     """
9     signal_color : str -> 'RED' | 'YELLOW' | 'GREEN' (from control room)
10    cx, cy       : float -> vehicle centroid (pixels)
11    speed        : float -> precomputed vehicle speed (m/s)
12    stop_line    : tuple -> (a, b, c) line coefficients
13    """
14    if signal_color != "RED":
15        return "NO_ANOMALY"
16
17    zone = get_zone(cx, cy, *stop_line)
18    if zone == "NORMAL":
19        return "NO_ANOMALY"
20
21    # Anomaly detected: vehicle is in the detected zone on red
22    if speed <= EPSILON:
23        return "STOP_LINE_VIOLATION"
24    else:
25        return "RED_LIGHT_VIOLATION"
26
27
28 def process_frame(signal_color, tracked_vehicles, stop_line):
29     """
30     tracked_vehicles : list of dicts, each with keys
31                       'id', 'cx', 'cy', 'speed'
32     Returns: dict {vehicle_id: violation_label}
33     """

```

```

34     results = {}
35     for v in tracked_vehicles:
36         results[v["id"]] = classify_violation(
37             signal_color, v["cx"], v["cy"], v["speed"], stop_line
38         )
39     return results
40
41
42 # --- Example usage ---
43 if __name__ == "__main__":
44     stop_line = (0, 1, -480)           # e.g. horizontal line y = 480
45     signal_color = "RED"
46     tracked_vehicles = [
47         {"id": 1, "cx": 320, "cy": 510, "speed": 0.0},    # past line, stopped
48         {"id": 2, "cx": 340, "cy": 540, "speed": 4.2},    # past line, moving
49         {"id": 3, "cx": 300, "cy": 400, "speed": 6.0},    # before line
50     ]
51
52     violations = process_frame(signal_color, tracked_vehicles, stop_line)
53     for vid, label in violations.items():
54         print(f"Vehicle {vid}: {label}")
55
56     # Output:
57     # Vehicle 1: STOP_LINE_VIOLATION
58     # Vehicle 2: RED_LIGHT_VIOLATION
59     # Vehicle 3: NO_ANOMALY

```

5 Important Logic Points

- **Signal color is externally supplied, not vision-inferred.** The system trusts the control room feed for S , eliminating signal-light-recognition failure modes (glare, occlusion, faded bulbs) as a source of false negatives/positives.
- **Red is the only gating condition.** No check is performed against Green or Yellow states – any non-red signal short-circuits the pipeline to “No anomaly” regardless of vehicle position.
- **Zone test acts as a spatial gate.** Only vehicles whose centroid crosses into Z_D while $S = \text{Red}$ are escalated; vehicles waiting correctly in Z_N are never flagged.
- **Speed is the sole discriminator between the two violation subtypes**, not an independent trigger – it only matters *after* an anomaly is already confirmed.
- **$v = 0$ does not imply “no violation.”** A stationary vehicle that has already crossed the stop line during red is still a violation (Stop-line violation), just a different category from an actively moving one.
- **Thresholding (ϵ) is necessary in practice** to avoid tracker/detection noise being misread as motion, since true $v = 0$ rarely occurs exactly in pixel-derived velocity estimates.
- **Per-vehicle, per-frame evaluation.** The decision is not global to the frame; each tracked vehicle is independently evaluated, allowing simultaneous detection of multiple violators.